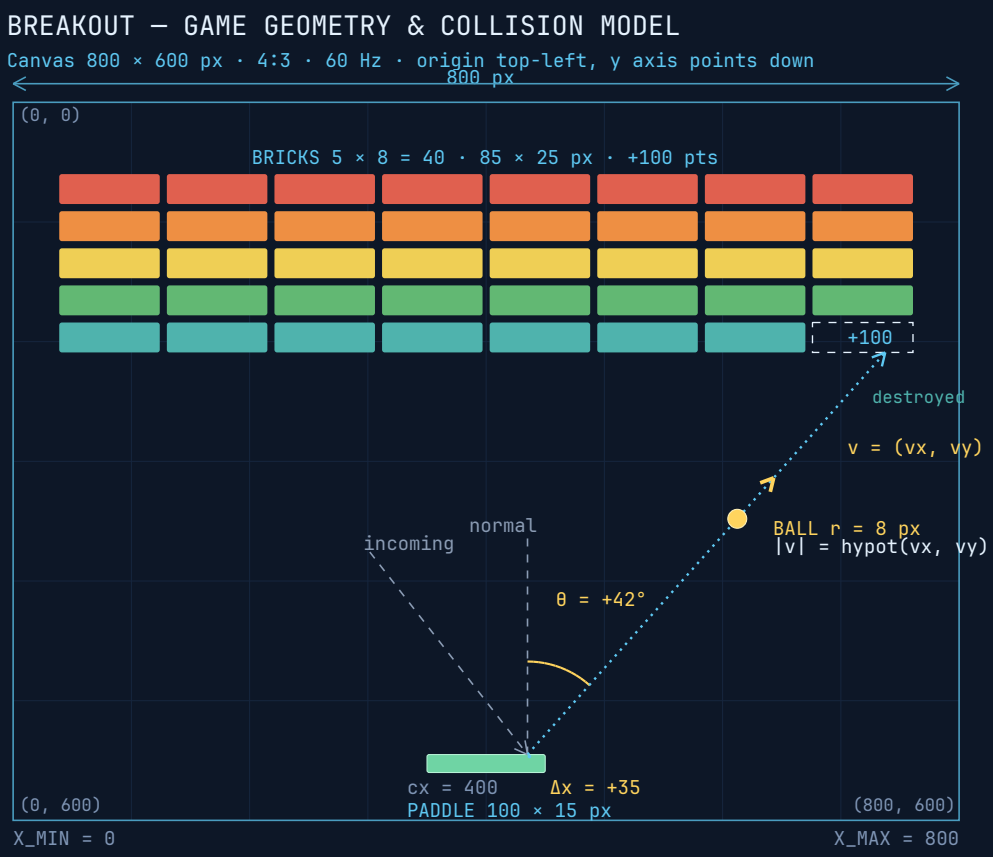
دانشگاه تهران
دانشکده علوم کامپیوتر

برنامه‌سازی پیشرفته

فاز پروژه

۰۱ هسته بازی Breakout

اهداف، نیازمندی‌های فنی، طراحی شی‌گرا و معیارهای ارزیابی




```
CANVAS
size    800 × 600 px (4:3)
tick    60 Hz
origin  top-left, y grows down

BRICKS
grid    5 rows × 8 cols = 40
size    85 × 25 px, gap 6
reward  +100 pts each

PADDLE
size    100 × 15 px
pos     y = 545, x in [0, 700]

BALL
radius  r = 8 px
speed   |v| = hypot(vx, vy)

PADDLE REFLECTION
offset = Δx / (w / 2)
clamped to [-1, 1]
θ = offset · 60°
vx = |v| · sin θ
vy = - |v| · cos θ
e.g. Δx = +35 → θ = +42°

CONTROLS
← →   move paddle
SPACE   serve
P       pause
R       reset
```

Dotted cyan = ball path after paddle hit · dashed grey = incoming path · yellow = velocity / angle

«در سال ۱۹۷۶، نولان باشنل در شرکت آتاری از استیو جابز خواست بازی Breakout را با کمترین تعداد تراشه سخت‌افزاری بسازد. جابز پروژه را به استیو وازنیاک سپرد؛ وازنیاک طی ۴ شبانه‌روز کار بی‌وقفه، مدار بازی را تنها با ۴۴ تراشه پیاده کرد. ابتکارات وازنیاک در بهینه‌سازی مدارهای گرافیکی این بازی، این تجربه را با طراحی نمایش تصویر و رنگ در رایانهٔ انقلابی Apple II مرتبط می‌دانند.»

فهرست مطالب

قوانین و شرایط تحویل ۲
معرفی بازی و اهداف ۳
بستر فنی و نسخهٔ جاوا ۴
اقدامات پیش از شروع ۵
ساختار و معماری سیستم ۶
موجودیت‌ها و هندسه بازی ۷
سازوکار گیم‌پلی و وضعیت‌ها ۸
گرافیک و واسط کاربری ۹
فیزیک و برخوردها ۱۰
گام‌های پیاده‌سازی ۱۲
معیارهای ارزیابی و ارائه ۱۳

قوانین و شرایط تحویل

پروژه درس برنامه‌سازی پیشرفته یک ماجراجویی فردی و جذاب است تا آموخته‌های شی‌گرایی را در قالب یک نرم‌افزار واقعی تجربه کنید. برای اینکه این مسیر یادگیری لذت‌بخش و عادلانه باشد، چند اصل ساده و حرفه‌ای را با هم مرور می‌کنیم:

استقلال در پیاده‌سازی و مالکیت کد

این پروژه کاملاً **فردی** است و هدفش ساختن اعتماد به نفس فنی شماست. تمام کدی که می‌نویسید باید حاصل درک مستقیم خودتان باشد. مطالعه منابع، راهنماها و مقالات برای یادگیری بسیار خوب است؛ اما کپی‌کردن کد دیگران یا ارسال کد تولیدشده بدون تسلط کامل، با اهداف آموزشی درس در تضاد است. در جلسه ارائه باید بتوانید فلسفه پشت هر خط کد را با تسلط کامل توضیح دهید.

ثبت پیوسته در گیت‌هاب

فرآیند تفکر مهندسی شما از طریق کامیت‌های مرحله‌به‌مرحله[۱] در مخزن گیت‌هاب[۲] سنجیده می‌شود. کدنویسی یک جریان آرام و پیوسته است؛ بنابراین قابلیت‌ها را گام‌به‌گام اضافه کرده و با پیام‌های تمیز ثبت کنید. از ارسال ناگهانی کل پروژه در قالب یک کامیت در ساعات پایانی بپرهیزید.

جلسه ارائه و گفت‌وگوی فنی

پس از تحویل مخزن در سامانهٔ ایلرن، در یک جلسه مصاحبه و ارزیابی دوستانه با دستیاران آموزشی، کد شما بررسی می‌شود. در این نشست، تسلط شما بر معماری، کلاس‌ها و نحوه پیاده‌سازی سنجیده می‌شود و در صورت نیاز نمرهٔ فنی تعدیل می‌گردد.

بستر تحویل و موعد نهایی

برای تحویل نهایی، نشانی مخزن گیت‌هاب پروژه را پیش از پایان موعد مقرر در سامانهٔ آموزشی ایلرن[۳] بارگذاری نمایید. دقت کنید که مخزن عمومی باشد یا دسترسی آن برای تیم تدریس باز شده باشد.

استفاده از ابزارهای هوش مصنوعی

هوش مصنوعی مثل یک منتور صبور است که می‌توانید برای فهم عمیق‌تر مفاهیم، اشکال‌زدایی و مشورت در طراحی از آن کمک بگیرید. با این حال، تصمیم‌گیرنده نهایی و مهندس اصلی سیستم شما هستید و تمام منطق بازی باید تحت تسلط کامل شما باشد.

همراهی دستیاران آموزشی

ما همیشه کنارتان هستیم تا مسیر یادگیری برایتان دلنشین‌تر شود. اگر در هر کجای معماری یا پیاده‌سازی با چالش یا ابهامی روبه‌رو شدید، در تالار گفتگو و جلسات رفع اشکال منتظر پرسش‌هایتان هستیم.

---

[۱] Git Commits
[۲] GitHub
[۳] Ilearn

معرفی بازی و اهداف

در سال ۱۹۷۶، شرکت آتاری با ساخت بازی تعاملی Breakout نقطه عطفی در تاریخ بازی‌های ویدئویی ساخت. داستان جذاب پشت این بازی، هنرنمایی استیو وازنیاک[۴] بود که مدار سخت‌افزاری آن را تنها با ۴۴ تراشه منطقی بست؛ روایت‌های رایج این تجربه را با طراحی معماری تصویر و رنگ رایانهٔ Apple II مرتبط می‌دانند.

امروز Breakout از دوست‌داشتنی‌ترین پروژه‌ها برای لمس واقعی مهندسی نرم‌افزار شی‌گرا و سیستم‌های تعاملی است: یک سکوی چابک در پایین صفحه، توپی پرانرژی که میان دیواره‌ها کمانه می‌کند، و دیواری از آجرهای رنگارنگ که منتظر نشانه‌گیری دقیق شما هستند.

اهداف اصلی فاز اول

در این فاز می‌خواهیم با هم یک هسته تمیز، پایدار و استاندارد به زبان جاوا[۵] بسازیم. مهم‌ترین مهارت‌هایی که در این مسیر تمرین می‌کنید:

۱. **طراحی شی‌گرا و تفکیک دغدغه‌ها:** جدا نگه‌داشتن دنیای ریاضی و قوانین برخورد بازی از دنیای نقاشی و رندرینگ گرافیکی.[۶]
۲. **اصل تک‌مسئولیتی:** سپردن یک وظیفه مشخص به هر کلاس و پرهیز از کلاس‌های همه‌کاره.[۷،۸]
۳. **کپسوله‌سازی شایسته:** مخفی‌کردن متغیرهای وضعیت و کنترل تغییرات از طریق متدهای شفاف.[۹]
۴. **مدیریت روان رخدادها:** گوش‌دادن به کلیدها و نوسازی پیوسته فریم‌ها در حلقه اصلی.[۱۰]
۵. **معماری منعطف برای آینده:** طراحی کلاس‌ها طوری که در فاز بعد ذخیرهٔ رکورد در فایل و پاورآپ‌ها به‌راحتی اضافه شود.

محدوده مورد انتظار در فاز اول

خیالتان راحت باشد. هدف این فاز سلامت ساختار کد، فیزیک درست برخوردها و روانی گیم‌پلی است. نیازی به منوهای پیچیده، صداگذاری، یا گرافیک‌های سه‌بعدی نیست؛ یک واسط بصری ساده، تمیز و روان بالاترین نمره را برای شما به همراه خواهد داشت.

---

[۴] Steve Wozniak
[۵] Java
[۶] SoC: Separation of Concerns
[۷] God Class
[۸] SRP: Single Responsibility Principle
[۹] Encapsulation
[۱۰] Event-Driven

بستر فنی و نسخهٔ جاوا

پیاده‌سازی فاز اول باید به زبان جاوا[۱۱] و با کتابخانهٔ گرافیکی قابل اجرا در محیط درس باشد. **مسیر رسمی درس:** ACM[۱۲] + Swing روی **Java 11**. بسترهای دیگر در صورت ثبت در README مجازند؛ پشتیبانی دستیاران در رفع اشکال برای آن‌ها محدودتر است.

بستر	نسخهٔ جاوا	یادداشت
ACM + Swing — مسیر رسمی	11 LTS	`acm.program.GraphicsProgram` و `acm.graphics.*`. فایل `acm.jar` را در classpath کامپایل و اجرا قرار دهید.
Swing خالص	11 تا 21	بدون ACM: `JPanel`، `Timer` و نقاشی دستی. مسئولیت بافر دوگانه[۱۳] با شماست.
JavaFX	17 یا 21	`Canvas` یا `Pane`. ماژول‌های JavaFX را خودتان به پروژه اضافه کنید.
Processing	17	حالت Java: `setup` و `draw`. معماری MVC را خودتان ساختاردهی کنید.

Java 8 نیز با همان `acm.jar` در بسیاری از محیط‌های درس کار می‌کند. JavaFX از JDK 11 به بعد جدا از JDK توزیع می‌شود و Processing 4 بر پایهٔ JDK 17 است.

آزادی در چیدمان و ظاهر

ابعاد، مختصات و رفتارهای ذکرشده در این سند برای موجودیت‌ها و فیزیک الزامی‌اند. رنگ آجرها، فونت پیام‌ها، چیدمان نوار وضعیت، نماد جان‌ها و هر جزئی بصری که عدد یا فرمول ندارد را آزادانه و سلیقه‌ای انتخاب کنید؛ فقط خوانایی و روانی ۶۰ هرتز را حفظ کنید.

---

[۱۱] Java

[۱۲] ACM: Java Task Force

[۱۳] Double Buffering

اقدامات پیش از شروع

پیش از نوشتن نخستین خطوط کد در جاوا، ۳ گام راهبردی زیر مسیر طراحی و توسعه را برایتان کاملاً روشن و سازمان‌یافته می‌کند:

۱. مخزن گیت‌هاب، گیت‌ایگنور و کامیت‌ها

پروژه حرفه‌ای با مدیریت نسخه استاندارد و ثبت تاریخچه تغییرات آغاز می‌شود. ۳ وظیفه کلیدی در این بخش:

ایجاد مخزن در گیت‌هاب — GitHub Repository

ساخت یک مخزن مستقل در گیت‌هاب به عنوان بستر اصلی نگه‌داری کد، رهگیری تغییرات در طول زمان و تحویل نسخه نهایی جهت ارزیابی در سامانهٔ ایلرن.

پیکربندی گیت‌ایگنور — `.gitignore`

فیلتر کردن فایل‌های خروجی کامپایل با پسوند `.class`، فایل‌های موقت سیستم‌عامل و تنظیمات محیط توسعه[۱۴] تا مخزن سبک و خوانا بماند.

کامیت‌های معنادار و استاندارد — Conventional Commits

ثبت پیوسته و گام‌به‌گام پیشرفت با پیشوندهای استاندارد مانند `feat:`، `fix:` و `refactor:` برای شفاف‌سازی سیر تفکر مهندسی شما.

۲. تجربه بازی و درک رفتار فیزیکی

برای درک عمیق مکانیک‌ها و ریتم بازی، چند دقیقه یک نسخه اصیل بریک‌اوت را تجربه کنید و این ۳ مشخصه را مد نظر قرار دهید:

زاویه کمانه توپ از سکو

توپ در برخورد با مرکز سکو بازتابی متمایل به عمود دارد و با نزدیک‌شدن نقطه برخورد به گوشه‌ها، زاویه پرتاب تندتر و افقی‌تر می‌شود.

روانی شبیه‌سازی در ۶۰ هرتز

پاسخ بی‌درنگ سکو به فشردن کلیدها و حرکت پیوسته و نرم توپ در سراسر صفحه بدون افت فریم، مکث یا پرش تصویری.

مکانیک اصابت و انهدام آجرها

حذف فوری آجر منهدم‌شده پس از اصابت، بازتاب لحظه‌ای توپ در صفحه و ثبت ۱۰۰ امتیاز به نفع بازیکن.

۳. مدل‌سازی شی‌گرا و نقشه کلاس‌ها

یک طراحی روشن پیش از کدنویسی، زمان پیاده‌سازی و اشکال‌زدایی را به حداقل می‌رساند:

ترسیم نقشه کلاس‌ها — اختیاری — UML Class Diagram

در صورت تمایل، پیش از آغاز کدنویسی کلاس‌ها، فیلدهای خصوصی و متدهای عمومی را در قالب یک نمودار کلاسی[۱۵] روی کاغذ یا با ابزار نرم‌افزاری ترسیم کنید تا تفکیک وظایف شفاف باشد. تحویل فایل نمودار در این فاز الزامی نیست.

نقش در جلسه ارائه

داشتن نقشه معماری، حتی روی کاغذ، در جلسه مصاحبه و ارزیابی به تسلط و استقلال شما در تفکیک مسئولیت‌ها کمک می‌کند؛ اما نمره‌ای مستقل برای تحویل نمودار در نظر گرفته نشده است.

---

[۱۴] IDE: Integrated Development Environment

[۱۵] UML: Unified Modeling Language

ساختار و معماری سیستم

تصور کنید تمام بازی در یک کلاس غول‌آسای ۲۵۰۰ خطی نوشته شود: از سرعت توپ و برخورد با دیواره تا رنگ صورتی آجرها و شنونده کلید Space. شاید اولش ساده به نظر برسد، اما با اولین باگ نامرئی، کد تبدیل به کلافی سردرگم می‌شود. نجات‌بخش ما در این بازی، الگوی درخشان معماری مدل-نما-کنترل‌گر[۱۶] است که وظایف را به ۳ بخش مستقل تقسیم می‌کند:

لایه مدل Model | کلاس‌های منطق

حقیقت جهان بازی؛ مختصات توپ، سکو، آجرها و جان‌ها. کدهای کاملاً مستقل از پنجره و گرافیک با آزمون‌پذیری آسان.

لایه نما BreakoutCanvas | View

پرده نقاشی و تماشا؛ دریافت وضعیت از کنترل‌گر و ترسیم اشکال، آجرها و نشانگر جان‌ها. بدون داشتن منطق بازی.

لایه کنترل‌گر GameEngine | Controller

مغز هماهنگ‌کننده؛ اجرای حلقه بازی در ۶۰ هرتز، دریافت فشرده‌شدن کلیدها، حل برخوردها و فرمان بازترسیم به نما.

جریان داده در ۳ گام

۱. **ورودی کاربر:** فشرده شدن کلیدهای جهت‌نما، Space، P و R توسط کنترل‌گر دریافت می‌شود.
۲. **محاسبه در مدل:** کنترل‌گر متدهای حرکتی سکو و توپ را صدا زده و وضعیت جدید دنیای بازی را ثبت می‌کند.
۳. **بازترسیم نما:** کنترل‌گر پس از قطعی شدن محاسبات فیزیک، وضعیت تازه را برای ترسیم به بوم نقاشی می‌فرستد.

قرارداد ذخیره رکورد: HighScoreStorage

برای استقلال کامل بازی از نحوه ذخیره‌سازی، یک رابط ساده تعریف می‌کنیم. در فاز اول پیاده‌سازی حافظه‌ای برای رکورد کافی است؛ اتصال فایل در همین فاز **اختیاری** است، اما هم در بخش امتیازی نمره دارد و هم برای فاز بعد و مباحث ورودی/خروجی آموزشی بسیار مفید است:

```
public interface HighScoreStorage {
    int loadHighScore();
    void saveHighScore(int score);
}
```

پیاده‌سازی موقت در حافظه InMemoryScoreStorage

نگه‌داری بالاترین رکورد در حافظهٔ موقت[۱۷] طی اجرای بازی؛ این پیاده‌سازی برای تحویل فاز اول الزامی است.

پیاده‌سازی آینده در فایل FileScoreStorage

ذخیره‌سازی دائمی رکورد در فایل متنی؛ می‌توانید همین فاز آن را پیاده کنید یا به فاز بعد موکول کنید—در هر صورت رابط HighScoreStorage ثابت می‌ماند.

---

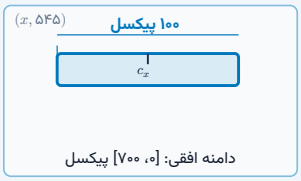
[۱۶] MVC: Model-View-Controller
[۱۷] RAM: Random Access Memory

موجودیت‌ها و هندسه بازی

جهان بازی Breakout از ۴ موجودیت بنیادین تشکیل شده که هر کدام مشخصات دقیق هندسی و مرزهای کپسوله‌شده داده‌ای دارند. مختصات صفحه با مبدا $(۰, ۰)$ در گوشه بالا-چپ بوم تعریف شده و تمام محاسبات بر پایه پیکسل انجام می‌گیرد:

سکو Paddle

ابعاد ۱۰۰×۱۵ پیکسل | Y=۵۴۵ پیکسل | بازه X: [۰, ۷۰۰]

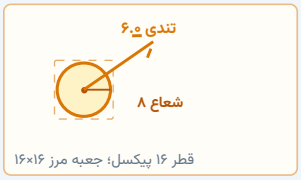


هندسه و ابعاد: مستطیل به ابعاد ۱۰۰ در ۱۵ پیکسل مستقر در فاصله ۴۰ پیکسلی از کف پنجره ($y = ۵۴۵$). بازه مجاز جابه‌جایی افقی $x \in [۰, ۷۰۰]$ است تا سکو هرگز از حاشیه چپ یا راست بوم خارج نشود.

رفتار و کپسوله‌سازی: فیلدهای مختصات با سطح دسترسی `private` هستند. جابه‌جایی با تندی ۸ پیکسل بر تیک فقط از طریق `moveLeft` و `moveRight` با مهار مرزی انجام می‌شود.

توپ Ball

قطر ۱۶ | شعاع ۸ | تندی پایه ۶.۰ پیکسل/تیک

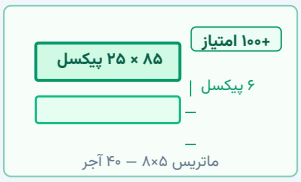


هندسه و ابعاد: دایره به قطر ۱۶ پیکسل؛ موقعیت با گوشه بالا-چپ جعبه مرز ۱۶×۱۶ ذخیره می‌شود. در برخورد با دیواره‌ها و آجرها، `reverseX` و `reverseY` مؤلفهٔ سرعت را معکوس می‌کنند. برخورد با سکو زاویه‌دار است و در بخش فیزیک توضیح داده شده، نه بازتاب آینه‌ای ساده.

رفتار و کپسوله‌سازی: در شروع دور، توپ روی سکو مهار است—مثلاً $(x_{\text{paddle}} + ۴۲, ۵۲۹)$ یعنی وسط سکو منهای شعاع توپ—و با `Space` پرتاب می‌شود. فیلدهای سرعت و موقعیت `private` هستند؛ برای سکو از `setVy/setVx` یا متد کمکی `bounceOffPaddle` استفاده کنید.

آجرها Brick

ماتریس ۴۰ آجر (۵×۸) | ابعاد ۸۵×۲۵ پیکسل | فاصله ۶ پیکسل

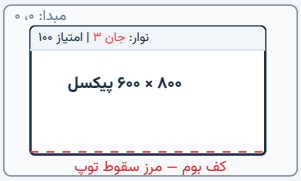


هندسه و ابعاد: شبکه ۵ سطر در ۸ ستون (۴۰ آجر). ابعاد هر آجر ۸۵ در ۲۵ پیکسل با فاصله ۶ پیکسل و حاشیه بالای ۶۰ پیکسل. فرمول مکان آجر سطر r و ستون c: $x = ۳۹ + c \times ۹۱$ و $y = ۶۰ + r \times ۳۱$.

رفتار و کپسوله‌سازی: هر آجر مستطیل مرز تداخل[۱۸] و وضعیت تخریب `private` دارد. متد `hit` با هر اصابت، آجر را نابود می‌کند و امتیاز ۱۰۰ را ثبت می‌کند.

نشانگر وضعیت و بوم HUD / Canvas

بوم ۸۰۰×۶۰۰ پیکسل | نوار وضعیت ارتفاع ۳۵ پیکسل



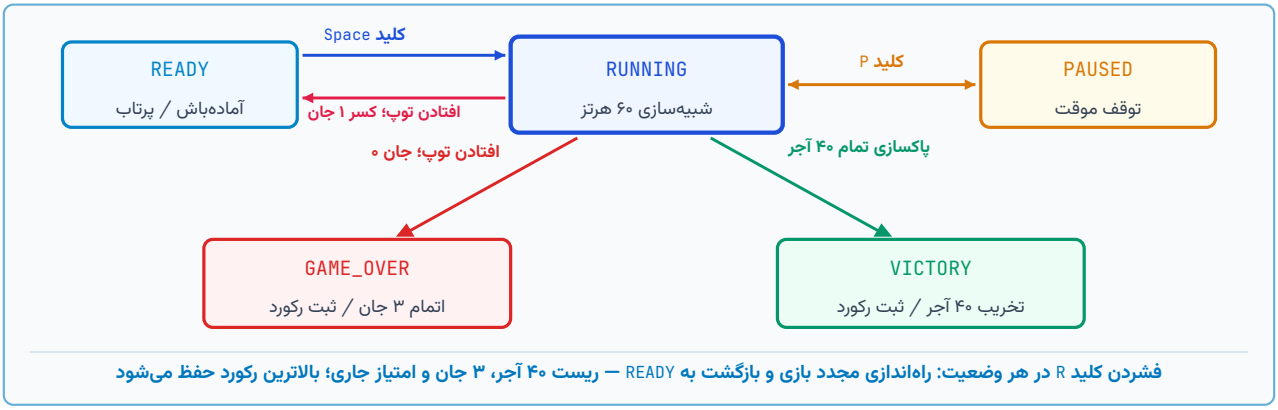
هندسه و ابعاد: پنجره گرافیکی با ابعاد ۸۰۰ در ۶۰۰ پیکسل و مبدا $(۰, ۰)$ در گوشه بالا-چپ. نوار بالایی صفحه به ارتفاع ۳۵ پیکسل برای نمایش وضعیت جان‌ها، امتیاز و رکورد بازی در نظر گرفته شده است.

رفتار و کپسوله‌سازی: موتور بازی ۳ جان و امتیاز را مدیریت می‌کند. وقتی لبهٔ پایین توپ از خط پایین بوم عبور کند، یک جان کم می‌شود؛ شرط دقیق در بخش گیم‌پلی آمده است. پس از اتمام جان‌ها یا تخریب ۴۰ آجر، نما پیام پایان را نشان می‌دهد.

[۱۸] AABB: Axis-Aligned Bounding Box

سازوکار گیم‌پلی و وضعیت‌ها

مدیریت جریان بازی با یک ماشین وضعیت متناهی و شفاف پیاده‌سازی می‌شود تا واکنش بازی به کلیدها، برخوردها و رویدادهای مرگ و پیروزی کاملاً ساختاریافته و پیش‌بینی‌پذیر باشد:



وضعیت	رفتار، شرایط ورود و انتقال‌های مجاز در جریان بازی
حالت آماده‌باش READY	وضعیت نخستین بازی یا پس از هر بار سوختن یک جان، در صورت داشتن جان باقی‌مانده. توپ روی مرکز سکو مستقر است و همراه با آن جابه‌جا می‌شود؛ فقط ورودی سکو و چسبیدن توپ پردازش می‌شود. با Space شبیه‌سازی شروع و بازی وارد وضعیت اجرا می‌شود.
حالت اجرای فعال RUNNING	حلقه اصلی شبیه‌سازی بلادرنگ ۶۰ هرتز؛ حرکت پیوسته توپ، فرمان‌پذیری سکو، بازتاب از دیواره‌ها و انهدام آجرها. وقتی لبهٔ پایین توپ از خط پایین بوم عبور کند، یک جان کم می‌شود و در صورت باقی‌ماندن جان به READY برمی‌گردید؛ با تخریب تمام آجرها وضعیت پیروزی فعال می‌شود.
توقف موقت PAUSED	با فشردن کلید P در حین بازی، محاسبات متوقف شده و پیام توقف روی صفحه ظاهر می‌شود. فشردن مجدد همین کلید شبیه‌سازی را دقیقاً از همان مختصات و با همان بردار سرعت پیشین ادامه می‌دهد.
پایان بازی / شکست GAME_OVER	وقتی لبهٔ پایین توپ از خط پایین بوم عبور کند و جان باقی نمانده باشد. شبیه‌سازی متوقف می‌شود، پیام پایان نمایش داده می‌شود و در صورت برتری امتیاز، بالاترین رکورد ثبت می‌شود. کلید R بازی را ریست می‌کند.
پیروزی بازیکن VICTORY	به محض انهدام کامل تمام ۴۰ آجر فعال می‌شود. حرکت توپ متوقف شده، پیام تبریک و امتیاز نهایی بازیکن نمایش داده شده و بالاترین رکورد ذخیره می‌گردد. فشردن کلید R امکان آغاز یک بازی جدید را فراهم می‌سازد.

پیاده‌سازی تمیز وضعیت‌ها با ساختار شمارشی enum

به جای متغیرهای بولی پراکنده مانند isPaused یا isGameOver، حالت بازی را با enum GameState و ۵ مقدار بالا مدل‌سازی کنید. در READY فقط ورودی سکو و مهار توپ را به‌روز کنید؛ محاسبات فیزیک و برخورد فقط در RUNNING پیش می‌روند. این تفکیک از باگ‌های منطقی رایج جلوگیری می‌کند.

گرافیک و واسط کاربری

نمایش بصری بازی بریک‌اوت بر پایه یک بوم دوبعدی شفاف و پایدار استوار است. رعایت چند اصل ساده گرافیک کامپیوتری باعث می‌شود بازی شما کاملاً روان و بدون پرش یا لرزش تصویر اجرا شود:

۱. دستگاه مختصات و هندسه بوم

در محیط‌های گرافیکی رایانه، جهت محورها با دستگاه دکارتی ریاضی کمی تفاوت دارد:

- نقطه مبدا با مختصات $(۰, ۰)$ در گوشه بالا-چپ بوم قرار گرفته است.
- محور افقی مثبت به سمت راست پنجره پیش می‌رود.
- محور عمودی مثبت به سمت پایین و کف پنجره امتداد دارد.

بنابراین هر زمان توپ رو به بالا و به سمت آجرها حرکت می‌کند، سرعت عمودی آن منفی است؛ و وقتی به سمت سکو در پایین سقوط می‌کند، سرعت عمودی مثبت است.

۲. پایداری تصویر و بافر دوگانه

شاید برایتان پیش آمده باشد که در برنامه‌های گرافیکی، تصویر چشمک بزند یا بلرزد. این پدیده به خاطر پاک‌شدن و کشیده‌شدن پیاپی اشیا روی صفحه است:

- با استفاده از بافر دوگانه[۱۹]، تمام نقاشی‌ها ابتدا در یک حافظه پس‌زمینه کامل می‌شوند و سپس فریم کامل‌شده یک‌باره به نمایش درمی‌آید.
- کتابخانه گرافیکی ACM این سازوکار را در بطن خود مدیریت می‌کند تا کار شما برای ایجاد انیمیشن روان و بی‌دردسر باشد.

۳. اجزای دیداری و نوار وضعیت

بخش بصری	مشخصات ترسیم و استانداردهای نمایش
بوم اصلی بازی	مستطیلی به ابعاد ۸۰۰ در ۶۰۰ پیکسل با پس‌زمینه تیره یا یکدست که محدوده شبیه‌سازی را مشخص می‌سازد.
سکو و توپ	سکو به صورت مستطیلی توپر با رنگ متمایز و توپ به شکل دایره‌ای یکدست با قطر ۱۶ پیکسل.
شبکه آجرها	مستطیل‌های رنگی هم‌اندازه با فاصله ۶ پیکسل بین آجرها؛ تنوع رنگ سطرها آزاد است.
نوار وضعیت[۲۰]	نوار بالای صفحه به ارتفاع ۳۵ پیکسل برای امتیاز، ۳ جان و رکورد؛ پیام‌های توقف و پایان در وسط بوم.

[۱۹] Double Buffering
[۲۰] HUD: Heads-Up Display

فیزیک و برخوردها

موتور فیزیک بازی وظیفه شبیه‌سازی حرکت پیوسته، محاسبه زاویه بازتاب از سکو و انهدام دقیق آجرها را بر عهده دارد. هر «تیک» یک گام از حلقهٔ ۶۰ هرتزی بازی است:

۱. بازتاب زاویه‌دار از سکو

اگر توپ همیشه مثل آینه بازتاب کند، بازی سریعاً وارد یک چرخه تکراری و خسته‌کننده می‌شود. برای پویایی بیشتر، زاویه پرتاب با توجه به فاصله نقطه برخورد تا مرکز سکو تعیین می‌گردد:

روابط ریاضی کمانه زاویه‌دار

- **۱. انحراف نسبی برخورد نسبت به مرکز سکو ($d \in [-1.0, +1.0]$):** فاصلهٔ افقی مرکز توپ تا مرکز سکو (Δx) و نیم‌عرض سکو (w_{half}) را به‌صورت متغیرهای تفکیک‌شده تعریف می‌کنیم:

$$\Delta x = x_{\text{ball}} - x_{\text{paddle_center}} \quad , \quad w_{\text{half}} = \frac{W_{\text{paddle}}}{2}$$

$$d = \frac{\Delta x}{w_{\text{half}}}$$

مقدار منفی نشان‌دهنده اصابت در نیمه چپ ($\Delta x < 0$) و مقدار مثبت نشان‌دهنده اصابت در نیمه راست سکو ($\Delta x > 0$) است.

- **۲. زاویه پرتاب متناسب با نقطه برخورد (تا بیشینه ۶۰ درجه):**

$$\theta = d \times 60^{\circ}$$

برخورد در مرکز سکو بازتابی عمودی ($\theta = 0$) و برخورد در لبه‌ها ($d = \pm 1.0$) تندترین زاویه پرتاب را می‌سازد.

- **۳. پایستگی تندی کل توپ بر پایه قضیه فیثاغورس:**

$$v = \sqrt{v_x^2 + v_y^2}$$

- **۴. تجزیه سرعت به مؤلفه‌های جدید (پرتاب رو به بالا):**

$$v_{x'} = v \sin(\theta) \quad , \quad v_{y'} = -|v \cos(\theta)|$$

علامت منفی در مؤلفه عمودی تضمین می‌کند که توپ همیشه رو به بالا و به سمت آجرها کمانه کند.

این قطعه فقط بازتاب زاویه‌دار از سکو است؛ برای دیواره‌ها و آجرها از `reverseX` و `reverseY` استفاده کنید.

```
// Calculate horizontal offset and half-width as separate variables
double paddleCenter = paddle.getX() + paddle.getWidth() / 2.0;
double ballCenter = ball.getX() + ball.getRadius();
double deltaX = ballCenter - paddleCenter;
double halfWidth = paddle.getWidth() / 2.0;

double offset = deltaX / halfWidth;
offset = Math.max(-1.0, Math.min(1.0, offset));

// Calculate bounce angle (max 60 deg) and preserve total velocity
double angle = offset * Math.toRadians(60.0);
double speed = Math.hypot(ball.getVx(), ball.getVy());

// Apply new velocity components (always deflect upwards)
ball.setVx(speed * Math.sin(angle));
ball.setVy(-Math.abs(speed * Math.cos(angle)));
```

۲. تشخیص برخورد با آجرها

برای بررسی برخورد توپ با آجرها و دیواره‌ها، از الگوریتم تداخل مستطیل‌های مرز[۲۱] استفاده می‌کنیم. به محض برخورد، محور برخورد را با عمق هم‌پوشانی تعیین کنید: عمق هم‌پوشانی توپ و آجر را در محور افقی و عمودی حساب کنید و فقط مؤلفهٔ سرعت محوری را معکوس کنید که عمق کمتری دارد. در گوشهٔ آجر نیز همین قاعده را به کار ببرید تا تنها یک مؤلفه معکوس شود. امتیاز ۱۰۰ با نابودی هر آجر ثبت می‌شود.

[۲۱] AABB: Axis-Aligned Bounding Box

نکات طلایی برای پیاده‌سازی روان فیزیک

۱. **گیر افتادن توپ در سکو:** اگر فقط علامت سرعت را منفی کنید، توپ ممکن است در فریم بعدی همچنان داخل سکو بماند و شروع به لرزیدن کند! چاره: پس از برخورد، موقعیت عمودی توپ را طوری تنظیم کنید که لبهٔ پایین جعبه مرز توپ مماس با لبهٔ بالای سکو باشد؛ مثلاً `ball.setY(paddle.getY() - 2 * ball.getRadius())` وقتی `getY` گوشه بالا-چپ است.

۲. **تقسیم صفر ناشی از اعداد صحیح:** در محاسبه نسبت فاصله، متغیرها باید از نوع اعشاری `double` باشند؛ در غیر این صورت تقسیم عدد صحیح حاصل را صفر می‌کند.

۳. **پرش شبح‌وار از موانع[۲۲]:** اگر سرعت توپ خیلی زیاد شود، ممکن است در یک فریم از عرض آجر رد شود بدون اینکه برخوردی ثبت شود! در هر تیک، توپ را کمتر از قطر خودش (۱۶ پیکسل) جابه‌جا کنید؛ در غیر این صورت ممکن است بدون ثبت برخورد از آجر عبور کند.

---

[۲۲] Tunneling

گام‌های پیاده‌سازی

برای اینکه قدم‌به‌قدم و با آرامش پیش بروید، پیشنهاد می‌کنیم ساخت بازی را به ۵ مرحله مستقل و لذت‌بخش تقسیم کنید:

۱. **گام اول: پنجره، بوم و سکوی بازی**
راه‌اندازی پنجره به ابعاد ۸۰۰ در ۶۰۰ پیکسل، ساخت کلاس سکو و کنترل حرکت افقی با کلیدهای چپ و راست.
معیار پذیرش: سکو به نرمی حرکت کند و با رسیدن به لبه‌های چپ و راست صفحه متوقف شود.

۲. **گام دوم: کلاس توپ، حرکت و بازتاب از دیواره‌ها**
ساخت کلاس توپ، بردار سرعت و بازتاب الاستیک از سه دیواره بالا، چپ و راست بوم.
معیار پذیرش: توپ در ابتدا همراه سکو حرکت کند و با زدن کلید فاصله، شلیک شود.

۳. **گام سوم: بازتاب زاویه‌دار از سکو و سیستم ۳ جان**
فرمول بازتاب زاویه‌دار بین منفی ۶۰ تا مثبت ۶۰ درجه و کسر جان وقتی توپ از سکو پایین‌تر رفت و لبهٔ پایین توپ از خط پایین بوم عبور کرد.
معیار پذیرش: با از دست رفتن توپ، یک جان کم شود و توپ دوباره روی سکو آماده پرتاب باشد.

۴. **گام چهارم: ماتریس ۴۰ آجر، تشخیص برخورد و امتیاز**
چیدمان آجرها در ۵ سطر و ۸ ستون، الگوریتم برخورد، ۱۰۰ امتیاز به ازای هر آجر نابودشده و نمایش بالاترین رکورد در حافظهٔ موقت.
معیار پذیرش: اصابت به هر آجر یک‌ضربه‌ای آن را محو کند و جهت توپ را تغییر دهد.

۵. **گام پنجم: ماشین وضعیت، توقف، برد، باخت و شروع دوباره**
اتصال وضعیت‌های بازی، پیام‌های توقف با کلید P و راه‌اندازی دوباره در هر لحظه با کلید R.
معیار پذیرش: بازی با از دست رفتن جان‌ها یا تخریب تمام آجرها به پایان برسد و کلید R همه‌چیز را ریست کند؛ بالاترین رکورد حفظ شود.

راهبرد توسعه و کامیت‌ها

پیشنهاد می‌کنیم بعد از تکمیل هر گام، بلافاصله یک کامیت با پیشوندهای Conventional Commits مانند `feat:` در گیت‌هاب ثبت کنید تا همیشه یک نسخهٔ پایدار در تاریخچه داشته باشید.

معیارهای ارزیابی و ارائه

فاز اول پروژه دارای ۱۰۰ نمره تخصصی است که معادل ۲ نمره از ۲۰ نمره کل پایانی درس محاسبه می‌شود. حداکثر ۱۵ نمرهٔ امتیازی اختیاری جدا از این سقف است:

معیار ارزیابی	مؤلفه‌ها و جزئیات بررسی فنی	بارم
درستی عملکرد و حس بازی	حرکت روان ۶۰ هرتز، پرتاب دستی با کلید Space، بازتاب زاویه‌دار از سکو، بازتاب الاستیک از سه دیواره، تخریب دقیق آجرها با الگوریتم تداخل مستطیل‌ها، چرخه ۳ جان و ماشین وضعیت جامع.	۳۵ نمره
معماری و طراحی شی‌گرا	تفکیک دقیق سه لایه مدل، نما و کنترل‌گر، کپسوله‌سازی کامل فیلدها با سطح دسترسی خصوصی، پایبندی به اصل تک‌مسئولیتی، و به‌کارگیری رابط قرارداد ذخیره‌سازی داده.	۳۵ نمره
کیفیت کد و مهندسی نرم‌افزار	پرهیز کامل از اعداد جادویی و تجمیع ثابت‌ها در یک کلاس مرجع، نام‌گذاری گویای متغیرها و توابع، ساختار استاندارد بسته‌بندی و تعهد به کامیت‌های منظم در گیت‌هاب.	۱۵ نمره
تسلط و گفت‌وگو در جلسه ارائه	تشریح روان محاسبات هندسی زاویه کمانه، تبیین منطق تشخیص برخورد، توجیه تصمیم‌های شی‌گرایی، پاسخ به پرسش‌های مفهومی و در صورت نیاز اعمال تغییر کوچک در جلسه.	۱۵ نمره
مجموع کل ارزیابی	**ارزیابی جامع فاز اول هسته بازی بریک‌اوت**	**۱۰۰ نمره**

یادداشت برای ارزیابان

شرط از دست رفتن توپ در پیاده‌سازی مرجع: عبور لبهٔ پایین جعبهٔ مرز ۱۶×۱۶ از خط $y = ۶۰۰$ پیکسل (همان شرط بخش گیم‌پلی) (لبهٔ پایین بوم بازی ۸۰۰×۶۰۰)، با قرارداد موقعیت گوشهٔ بالا-چپ. پیاده‌سازی‌های معادل با همان هندسه پذیرفته می‌شوند.

جزئیات جلسه ارائه

جلسه ارائه، پس از تحویل مخزن، مصاحبهٔ خشک نیست؛ گفت‌وگوی صمیمی و دوستانه با دستیاران آموزشی است تا دستاوردها و تلاش‌هایتان را نشان دهید و بازخوردهای سازنده بگیرید. اگر کدها را خودتان نوشته باشید و چرایی تصمیم‌هایتان را بدانید، این جلسه بسیار لذت‌بخش خواهد بود:

- **تسلط کامل بر کدها:** اگر ساختار کدهای ارسالی را خوب بشناسید و منطق محاسبات فیزیک و کلاس‌ها را روان توضیح دهید، می‌توانید ۱۵ نمرهٔ این ردیف را کامل بگیرید.
- **اصالت کار و استقلال:** هدف ما اطمینان از یادگیری واقعی شماست. اگر نتوانید دلیل تصمیم‌های کدتان را توضیح دهید یا تغییرات کوچکی را در جلسه اعمال کنید، نمرهٔ فنی با ضریب α تعدیل می‌شود. دستیاران آموزشی پس از جلسه، میزان تسلط شما را با عددی از ۰ تا ۱۰۰ (یعنی m) می‌سنجند و ضریب را این‌گونه محاسبه می‌کنند: $\alpha = 0.5 + 0.5 \cdot \frac{m}{100}$. نمرهٔ نهایی فاز برابر است با نمرهٔ فنی ضربدر α. برای مثال، اگر نمرهٔ فنی ۸۰ و میزان تسلط ۶۰ باشد، آنگاه $\alpha = ۰/۸$ و نمرهٔ نهایی ۶۴ خواهد بود. با این فرمول همواره $0.5 \le \alpha \le 1$ است؛ یعنی کمترین ضریب ۰٫۵ برای تسلط صفر و بیشترین ضریب ۱ برای تسلط کامل. هوش مصنوعی دستیار خوبی برای یادگیری است، اما مهندس اصلی پروژه خود شمایید.

بخش‌های امتیازی و اختیاری

دو قابلیت اختیاری با بارم مشخص؛ جمع امتیازی‌ها حداکثر ۱۵ نمره و جزو سقف ۱۰۰ نمرهٔ پایه نیست:

- **آجرهای مقاوم چندضربه‌ای — ۵ نمره امتیازی:** آجرهایی که با ضربهٔ اول فقط آسیب می‌بینند و با ضربهٔ بعد منهدم می‌شوند؛ امتیاز بازی فقط هنگام نابودی کامل تعلق می‌گیرد، نه در هر اصابت.
- **تعویض پویای راهبرد ذخیره‌ساز — ۱۰ نمره امتیازی:** تغییر بلادرنگ نوع ذخیره رکورد میان حافظه موقت و فایل متنی در حین اجرای بازی.